

Objective

To create a scenario where a child process becomes a zombie process, investigate the process table, and modify the program to eliminate zombie processes using proper synchronization techniques.

Part 1 - Creating a Zombie Process

The program `zombie.c` creates a child process using `fork()`.

The child process terminates using `exit(0)`, while the parent process does not call `wait()` or `waitpid()`.

The parent sleeps for 120 seconds, allowing the terminated child to remain in the process table as a zombie.

Compilation

```
gcc zombie.c -o zombie
./zombie
```

Process Table Investigation

The process table was investigated using the `ps` command.

Command: `ps -p 599 -o pid,ppid,state,cmd`

Observed output:

PID	PPID	S	CMD
599	598	Z	[zombie] <defunct>

Observation

PID 599 is the child process and PID 598 is its parent process.

The state Z indicates that the child process is in the zombie state.

The <defunct> status indicates that the child process has terminated but its parent has not yet collected its `exit` status.

Thus, a zombie process was successfully created and observed.

Part 2: Eliminating the Zombie Process

To eliminate the zombie process, the program was modified to use the `waitpid()` system call.

The parent process uses the following statement:

```
waitpid(pid, &status, 0);
```

This allows the parent to wait for the child process and collect its termination status.

Compilation

```
gcc zombie_fixed.c -o zombie_fixed
./zombie_fixed
```

Parent process:

PID = <parent PID>

Child PID = <child PID>

Parent waiting for child...

Child process:

PID = <child PID>

PPID = <parent PID>

Child exiting...

Child has been successfully reaped.

Parent exiting...

Verification

The process table was checked using:

```
ps -o pid,ppid,state,cmd | grep '[z]ombie_fixed'
```

No zombie process remained after the child process was reaped.

Conclusion

A zombie process was successfully created by allowing a child process to terminate without the parent collecting its `exit` status.

The zombie process was identified using the `ps` command, where its state was shown as Z and it was displayed as <defunct>.

The program was then modified to use `waitpid()`. The parent process properly synchronized with the child and collected its termination status.

Therefore, the zombie process was successfully eliminated using proper parent-child synchronization.

```
#include <stdio.h>

#include <stdlib.h>

#include <unistd.h>

int main()
{
    pid_t pid = fork();

    if (pid < 0)
    {
        perror("fork failed");
        return 1;
    }

    if (pid == 0)
    {
        printf("Child process:\n");
        printf("PID = %d\n", getpid());
        printf("PPID = %d\n", getppid());

        printf("Child exiting...\n");
        exit(0);
    }
    else
    {
        printf("Parent process:\n");
```

```
printf("PID = %d\n", getpid());  
printf("Child PID = %d\n", pid);  
  
printf("Parent sleeping for 120 seconds...\n");  
printf("During this time, the child becomes a zombie.\n");  
  
sleep(120);  
  
printf("Parent exiting...\n");  
}  
  
return 0;  
}  
  
#include <stdio.h>  
#include <stdlib.h>  
#include <unistd.h>  
#include <sys/wait.h>  
  
int main()  
{  
    pid_t pid;  
    int status;  
  
    pid = fork();  
  
    if (pid < 0)  
    {
```

```
perror("fork failed");

return 1;
}

if (pid == 0)
{
    printf("Child process:\n");
    printf("PID = %d\n", getpid());
    printf("PPID = %d\n", getppid());

    printf("Child exiting...\n");
    exit(0);
}
else
{
    printf("Parent process:\n");
    printf("PID = %d\n", getpid());
    printf("Child PID = %d\n", pid);

    printf("Parent waiting for child...\n");

    waitpid(pid, &status, 0);

    printf("Child has been successfully reaped.\n");
    printf("Parent exiting...\n");
}

return 0;
```

```
}
```

Output-

PRACTICAL-04 SECOND PART

ZOMBIE PROCESS EXPERIMENT

=====

OBJECTIVE

To create a zombie process, investigate the process table, and eliminate the zombie process using proper synchronization techniques.

PART 1: CREATING A ZOMBIE PROCESS

=====

Program:

zombie.c

Compilation Command:

gcc zombie.c -o zombie

Execution Command:

./zombie

The child process was created using fork().

The child process terminated using exit(0), while the parent process continued running for 120 seconds.

The parent did not call wait() or waitpid(), so the terminated child remained in the process table as a zombie.

PROCESS TABLE INVESTIGATION

=====

Command:

```
ps -p 599 -o pid,ppid,state,cmd
```

Output:

PID	PPID	S	CMD
-----	------	---	-----

599	598	Z	[zombie] <defunct>
-----	-----	---	--------------------

OBSERVATION

PID 599 is the child process.

PID 598 is the parent process.

The state Z indicates that the child process is a zombie.

The <defunct> status indicates that the child process has terminated but the parent has not yet collected its exit status.

Therefore, a zombie process was successfully created and observed.

PART 2: ELIMINATING THE ZOMBIE PROCESS

=====

Program:

zombie_fixed.c

Compilation Command:

gcc zombie_fixed.c -o zombie_fixed

Execution Command:

./zombie_fixed

Output:

Parent process:

PID = [actual parent PID]

Child PID = [actual child PID]

Parent waiting for child...

Child process:

PID = [actual child PID]

PPID = [actual parent PID]

Child exiting...

Child has been successfully reaped.

Parent exiting.

SYNCHRONIZATION USED

=====

The parent process uses `waitpid()` to wait for the child:

```
waitpid(pid, &status, 0);
```

The `waitpid()` system call allows the parent to collect the child's termination status and properly reap the child process.

VERIFICATION OF FIXED PROGRAM

=====

Command:

```
ps -o pid,ppid,state,cmd | grep '[z]ombie_fixed'
```

Result:

No zombie_fixed process remained after the child process was reaped.

COMPARISON

=====

zombie.c:

Child terminates

|

v

Parent does not call wait()

|

v

Child becomes a zombie

|

v

ps shows state Z

zombie_fixed.c:

Child terminates

|

v

Parent calls waitpid()

|

v

Child is reaped

|

v

No zombie remains

CONCLUSION

=====

A zombie process was successfully created by allowing a child process to terminate without the parent collecting its exit status.

The zombie process was identified using the ps command. Its state was shown as Z and it appeared as <defunct>.

The program was then modified to use waitpid(). The parent properly synchronized with the child and collected its

termination status.

Therefore, the zombie process was successfully eliminated using proper parent-child synchronization.